De Casteljau Algorithm

Quadratic Bézier — CairoMakie Visualisation

Code and Plot Explanation for Julia Beginners

`12-de-casteljau-makie.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `12-de-casteljau-makie.jl`, the CairoMakie counterpart to `12-de-casteljau-plots.jl`. The De Casteljau algorithm, all data structures, and all computed values are identical to the Plots.jl version. This document is fully self-contained but focuses on the API differences from Plots.jl. The math section is kept brief — refer to `12-de-casteljau-plots-explained` for the full geometric explanation and all data tables.

Contents

1 The Plot **2**

2 Visual Differences from the Plots.jl Version **2**

3 What Is Identical to the Plots.jl Version **3**

4 The Math (Brief Summary) **3**

5 `draw_and_save` — The Makie Drawing API **3**

5.1 Figure and Axis: `titlegap` 3
5.2 Curve and Control Polygon 4
5.3 Control Point Labels 4
5.4 The Snapshot Loop 4
5.5 Legend and Save 5

6 Entry Point **5**

7 Complete API Reference: This Script vs Plots.jl **6**

1. The Plot

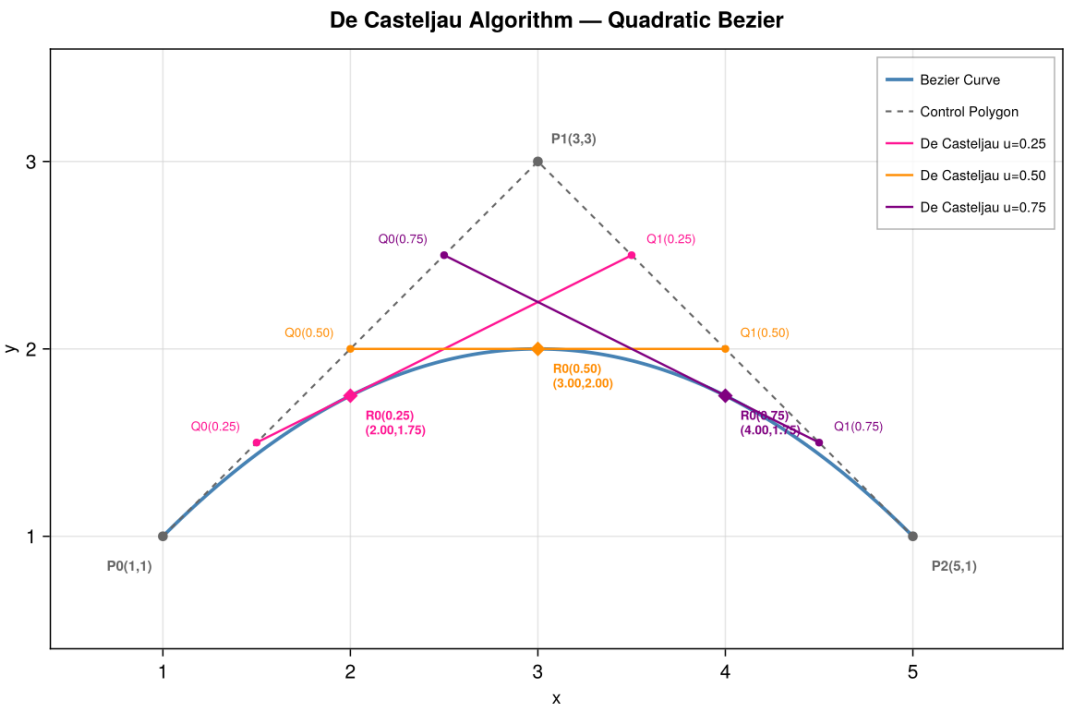

Figure 1: De Casteljau algorithm at $u = 0.25$ (pink), $u = 0.50$ (orange), $u = 0.75$ (purple). Rendered with CairoMakie.

2. Visual Differences from the Plots.jl Version

Element	Plots.jl	CairoMakie
Frame	Four-sided box	Open axes
Title font	Normal weight	**Bold**, `titlesize=16`
Title spacing	No gap control	`titlegap=12` adds breathing room
Control polygon	Gray30	Gray40 (slightly lighter)
Control markers	Size 6	Size 10 (more prominent)
Q dots	Size 5	Size 8
R_0 diamond	Size 7	Size 12 (clearly visible)
Resolution	800×600 px	1200×900 px (`px_per_unit=1.5`)
Legend	`legend=:topright` in `plot()`	`axislegend(ax)`

The Makie version has consistently larger markers and a bolder title, giving the construction points more visual weight against the higher-resolution canvas.

3. What Is Identical to the Plots.jl Version

Component	Note
`CONTROL_POINTS`, `U_SNAPSHOTS`, `SNAPSHOT_COLORS`	Byte-for-byte identical.
`DeCasteljauQuadratic` struct & constructor	Identical.
`lerp`	Identical one-liner broadcast function.
`steps`	Identical — same three `lerp` calls, same nam return.
`dc_curve`	Identical — same `.level2` field access patte
Colour constants	`COLOR_CURVE = :steelblue`; `COLOR_PO` `:gray40` (vs `:gray30` in Plots.jl).
`CTRL_NAMES`, `CTRL_OFFSET`, `Q_OFFSETS`	Identical.
All data tables	`l0`, `l1`, `r0` values are the same. `12-de-casteljau-plots-explained` for bles.

4. The Math (Brief Summary)

The De Casteljau algorithm evaluates a Bézier curve at parameter u by two rounds of linear interpolation.

Level 1: $Q_0 = (1-u)P_0 + uP_1, \quad Q_1 = (1-u)P_1 + uP_2$

Level 2: $R_0 = (1-u)Q_0 + uQ_1 \quad$ (the curve point)

The `lerp` function computes one linear interpolation. `steps(dc, u)` calls `lerp` three times and returns the three levels as a named tuple. See `12-de-casteljau-plots-explained` for the full geometric explanation and worked examples at all three u values.

5. `draw_and_save` — The Makie Drawing API

5.1. Figure and Axis: `titlegap`

```
fig = Figure(size = (800, 600), backgroundcolor = :white)

ax = Axis(fig[1, 1],
    title          = "De Casteljau Algorithm - Quadratic Bezier",
    titlesize      = 16,
    titlegap       = 12,
    xlabel         = "x",
    ylabel         = "y",
    xlabelsize     = 13,
    ylabelsize     = 13,
    limits         = (0.4, 5.8, 0.4, 3.6),
    aspect         = DataAspect(),
```

```
    xgridcolor      = (:lightgray, 0.6),
    ygridcolor      = (:lightgray, 0.6),
    backgroundcolor = :white,
)
```

Most attributes here were explained in `11-bezier-quadratic-makie-explained`. The one new attribute is `titlegap`.

`titlegap = 12`: Controls the vertical gap (in points) between the title text and the top edge of the plot area. Without this, Makie places the title very close to the axis frame, which can look cramped when the title is large (`titlesize = 16`). A value of 12 adds comfortable breathing room.

> **Plots.jl** has no direct equivalent to `titlegap`. The title spacing is handled automatically by the layout engine and cannot be fine-tuned per-plot.
>
> **CairoMakie** exposes `titlegap` as an explicit `Axis` attribute, giving you pixel-level control.

5.2. Curve and Control Polygon

```
# Bezier curve
cpts = dc_curve(dc)
lines!(ax, cpts[:,1], cpts[:,2],
    color = COLOR_CURVE, linewidth = 2.5, label = "Bezier Curve")

# Control polygon
l0 = steps(dc, 0.0).level0
lines!(ax, l0[:,1], l0[:,2],
    color = COLOR_POLYGON, linestyle = :dash,
    linewidth = 1.4, label = "Control Polygon")
scatter!(ax, l0[:,1], l0[:,2],
    color = COLOR_POLYGON, markersize = 10, strokewidth = 0)
```

Identical in logic to the Plots.jl version. Makie requires two separate calls (`lines!` and `scatter!`) for the control polygon line and its markers, as established in script 11.

The control polygon markers are `markersize = 10` here (vs 6 in Plots.jl), producing the larger grey dots visible at P_0, P_1, P_2 in the Makie plot.

5.3. Control Point Labels

```
for (i, (name, (dx, dy))) in enumerate(zip(CTRL_NAMES, CTRL_OFFSET))
    text!(ax, l0[i,1]+dx, l0[i,2]+dy,
        text     = name,
        color    = COLOR_POLYGON,
        fontsize = 10,
        font     = :bold,
    )
end
```

The loop pattern — `enumerate(zip(names, offsets))` — is identical to the Plots.jl version.

One small difference: the label colour here is `COLOR_POLYGON` (`:gray40`), giving labels the same grey as the control polygon. In the Plots.jl version, `COLOR_CTRL = :crimson` made the labels red like the polygon markers. In this Makie version the control polygon is grey, so the labels match that grey.

5.4. The Snapshot Loop

```
for (u, color) in zip(u_snapshots, colors)
    s  = steps(dc, u)
    l1 = s.level1
    r0 = s.level2

    # Level-1 segment
    lines!(ax, l1[:,1], l1[:,2],
        color = color, linewidth = 1.6,
        label = @sprintf("De Casteljau u=%.2f", u))

    # Level-1 Q dots
    scatter!(ax, l1[:,1], l1[:,2],
        color = color, markersize = 8, strokewidth = 0)

    # Level-2 R0 diamond
    scatter!(ax, [r0[1]], [r0[2]],
        color = color, marker = :diamond,
        markersize = 12, strokewidth = 0)

    # Q0, Q1 labels
    for (j, (qname, (dx, dy))) in enumerate(zip(["Q0","Q1"], Q_OFFSETS))
        text!(ax, l1[j,1]+dx, l1[j,2]+dy,
            text = @sprintf("%s(%.2f)", qname, u),
            color = color, fontsize = 9)
    end

    # R0 label
    text!(ax, r0[1]+0.08, r0[2]-0.22,
        text  = @sprintf("R0(%.2f)\n(%.2f,%.2f)", u, r0[1], r0[2]),
        color = color, fontsize = 9, font = :bold)
end
```

The loop structure and logic are identical to the Plots.jl version. The API differences follow the established Makie pattern:

Operation	Plots.jl	CairoMakie
Draw line segment	`plot!(plt, ...)`	`lines!(ax, ...)`
Draw scatter dots	`scatter!(plt, ...)`	`scatter!(ax, ...)`
Suppress legend entry	`label = false`	*(no label keyword passed)*
Place text	`annotate!(plt, x, y, text(...))`	`text!(ax, x, y, text=...)`

Suppressing legend entries in Makie. In Plots.jl, `label = false` explicitly hides a layer from the legend. In Makie, simply omitting the `label` keyword from a `scatter!` or `lines!` call is sufficient — unlabelled elements never appear in the legend. This is why the Q dots and R_0 diamonds in this script have no `label` argument at all.

The `[r0[1]]` single-point scatter idiom (same as Plots.jl):

`r0 = s.level2` is a length-2 vector, e.g. `[2.0, 1.75]`.

`scatter!(ax, [r0[1]], [r0[2]], ...)` wraps each coordinate in a length-1 vector to satisfy Makie's requirement that x and y arguments are arrays, not scalars.

Same pattern as the Plots.jl version — both libraries require arrays for scatter coordinates.

5.5. Legend and Save

```
axislegend(ax,
    position        = :rt,
    labelsize       = 10,
    framecolor      = (:black, 0.3),
    backgroundcolor = (:white, 0.8),
)

save(filename, fig, px_per_unit = 1.5)
println("Saved: $filename")
display(fig)
println("Press Enter to close...")
readline()
```

Identical to `11-bezier-quadratic-makie-explained`. `axislegend` reads the five labelled elements (Bezier Curve, Control Polygon, and three De Casteljau snapshots) and builds the legend automatically. The unlabelled `scatter!` calls for dots and diamonds are correctly excluded.

6. Entry Point

```
function main()
    dc = DeCasteljauQuadratic(CONTROL_POINTS)
    draw_and_save(dc, U_SNAPSHOTS, SNAPSHOT_COLORS,
                  "12-bezier_decasteljau-makie.png")
end

main()
```

Identical structure to the Plots.jl version.

How to run:

```
julia 12-de-casteljau-makie.jl
```

Saves `12-bezier_decasteljau-makie.png` and opens a window. Press Enter to close.

7. Complete API Reference: This Script vs Plots.jl

Operation	`12-de-casteljau-plots.jl`	`12-de-casteljau-makie.jl`
Create canvas	`plt = plot(...)`	`fig = Figure(...)`
Create axes	(combined with canvas)	`ax = Axis(fig[1,1], ...)`
Title gap	(not available)	`titlegap = 12`
Draw line	`plot!(plt, xs, ys)`	`lines!(ax, xs, ys)`
Draw markers	`scatter!(plt, xs, ys)`	`scatter!(ax, xs, ys)`
Suppress legend	`label = false`	(omit `label` keyword)
Text annotation	`annotate!(plt, x, y, text(...))`	`text!(ax, x, y, text=...)`
Single point	`scatter!(plt, [x], [y])`	`scatter!(ax, [x], [y])`
Legend	`legend=:topright` in `plot()`	`axislegend(ax, position=:rt)`
Save	`savefig(plt, filename)`	`save(filename, fig, px_per_unit=1.5)`

Explanation for `12-de-casteljau-makie.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated